CSCI 430 Project 2 Implementation Stage 1

Person 1 Class Designs

Group 9

Abel Asfaw, Connor Doolittle, Muhammad Irfan Younas, Ethan Manninen

Person 1: Muhammad Irfan Younas Due October 25, 2026

This part designs the opening menu and the shared FSM navigation. The interface has four menu states. Warehouse and the other Project 1 backend classes remain unchanged.

WarehouseContext

WarehouseContext corresponds to LibContext in the sample. It owns the state objects, transition matrix, active client ID, and original login role. Only Context chooses the next state.

Fields

private static WarehouseContext context; Singleton Context instance.

private Warehouse warehouse; Reference obtained through Warehouse.instance().

private int currentState; Index of the active menu, initially OPENING.

private int loginRole; Original role: NONE, CLIENT, CLERK, or MANAGER.

private String clientID; Validated active client ID; null outside the client menu.

private WarehouseState[] states; Four singleton menu objects indexed by state constants.

private int[][] nextState; Centralized state by event transition matrix.

private boolean running; Controls the main dispatch loop.

private BufferedReader reader; One shared reader for System.in.

State constants (public static final int): OPENING = 0, CLIENT = 1, CLERK = 2, MANAGER = 3. Role NONE = -1. Event constants (private static final int): TO_OPENING = 0, TO_CLIENT = 1, TO_CLERK = 2, TO_MANAGER = 3, EXIT = 4. Matrix values INVALID = -2 and STOP = -1 are control markers, not extra menu states.

Construction and access

private WarehouseContext(); Connects to Warehouse, initializes the reader, state array and matrix, and starts at OPENING with loginRole NONE and no client ID.

public static WarehouseContext instance(); Returns the shared Context.

public Warehouse getWarehouse(); Returns the existing Warehouse facade.

public String getClientID(); Returns the active client ID for client operations.

References

LibraryFSM template: github.com/rsarnath/LibTextFSM, samplecode/LibTextFSM (LibContext, LibState, Loginstate). Backend: abelSam10/430Project1FinalGroup9, WarehouseFinal/Warehouse.java. Requirements: Project 2 Implementation Stage 1 Fall 26 handout.

WarehouseContext methods

public void process(); While running, calls states[currentState].run(). Each run returns after a successful role transition. Dispatch is iterative so switching menus does not build nested run calls.

public static void main(String[] args); Starts WarehouseContext.instance().process().

public String getToken(String prompt); Reads a trimmed, nonempty line. Repeats for blank input. End of input or input failure sets running to false and returns null; the caller returns from its menu.

public boolean login(int role, String id); Allowed only in OPENING. Accepts CLIENT, CLERK, or MANAGER. CLIENT requires warehouse.getClient(id) != null. Clerk and manager do not require an ID. On success stores the original role and any client ID, then applies the matching matrix event; otherwise returns false without changing session fields.

public boolean becomeClient(String id); Allowed only in CLERK. Validates id with warehouse.getClient(id). On success stores the ID and transitions to CLIENT while preserving loginRole. Returns false without changing the session for an invalid request.

public boolean becomeClerk(); Allowed only in MANAGER. Preserves the original manager login role and transitions to CLERK. Returns false for any other source state.

public void logout(); Resolves the return event using currentState and loginRole, clears the client ID when leaving CLIENT, and applies the matrix. Returning to OPENING also resets loginRole to NONE.

public void exit(); Allowed from OPENING only. Applies EXIT and stops the dispatch loop. It does not require a fifth state.

private boolean changeState(int event); Checks the indexes and reads nextState[currentState][event]. INVALID leaves state and session unchanged. STOP sets running to false. Otherwise updates currentState and returns true. It never calls another state recursively.

Centralized transition matrix

Columns are Context events, not the local command numbers used by each menu. Navigation methods above enforce the role and source-state conditions before applying an event.

State	TO_OPENING	TO_CLIENT	TO_CLERK	TO_MANAGER	EXIT
OPENING	INVALID	CLIENT	CLERK	MANAGER	STOP
CLIENT	OPENING	INVALID	CLERK	INVALID	INVALID
CLERK	OPENING	CLIENT	INVALID	MANAGER	INVALID
MANAGER	OPENING	INVALID	CLERK	INVALID	INVALID

Logout rules: CLIENT returns to OPENING for a direct client login; otherwise it returns to CLERK. CLERK returns to MANAGER for an original manager login; otherwise it returns to OPENING. MANAGER returns to OPENING. Logout is not an operation of OPENING.

OpeningState extends WarehouseState

OpeningState corresponds to Loginstate in the sample. It displays the opening choices and collects the client ID when needed. It does not contain a transition matrix or decide delegated logout destinations.

Fields and methods

private static OpeningState state; Singleton opening menu.

private static final int CLIENT_LOGIN = 0; Opening command for a client.

private static final int CLERK_LOGIN = 1; Opening command for a clerk.

private static final int MANAGER_LOGIN = 2; Opening command for a manager.

private static final int EXIT = 3; Opening command to leave the program.

private OpeningState(); Constructs the menu without requesting Context during singleton construction.

public static OpeningState instance(); Returns the shared opening menu.

private int getCommand(); Uses Context.getToken to read a command. Repeats for nonnumeric input or a number outside 0 through 3. Returns -1 on end of input; run then returns.

public void run(); Obtains Context after initialization. Displays choices and reads commands. For client login, prompts for an ID and calls context.login(CLIENT, id). Clerk and manager commands call login with the corresponding role and null ID. A successful login returns to the Context dispatch loop. An invalid ID displays an error and stays in this menu. Exit calls context.exit() and returns.

WarehouseState

This abstract superclass corresponds to LibState. OpeningState, ClientMenuState, ClerkMenuState, and ManagerMenuState extend it. The other three menu designs belong to the other group members.

protected WarehouseContext context; Shared Context reference assigned when a menu runs, after Context initialization.

protected WarehouseState(); Common constructor; does not call WarehouseContext.instance().

public abstract void run(); Displays the menu and handles operations until a successful transition or termination. Business operations finish in the same menu.

Shared navigation agreement

Client operations read context.getClientID() rather than requesting another client ID. Clerk calls context.becomeClient(id); manager calls context.becomeClerk(); all three role menus call context.logout(). Each menu returns immediately after successful navigation. Menu classes do not directly call another menu or change session fields.

For a manager session the required route is OPENING to MANAGER to CLERK to CLIENT, then CLIENT to CLERK to MANAGER to OPENING. A clerk session returns CLIENT to CLERK to OPENING. A direct client session returns CLIENT to OPENING. These fixed routes need the original login role, not a stack of previous menus.

Design checks: reject an unknown client ID without changing roles; clear the previous client ID on logout; allow a clerk to select C1 and later C2; reject invalid commands without leaving the menu; and verify each return route above. No password classes or extra dialog states are added for this text-only stage.